

Navigation in ROS 2 & Gazebo

###NOTE: [at the end of the file](#) is the list of functions/ROS&GZ packages which we can use for our trace planning

+++++

MEGA SHORT SUMMARY FOR U

1. **Perception** → Sensors provide data.
2. **Localization** → The robot determines its position.
3. **Mapping** → The environment is understood.
4. **Path Planning** → A safe route is calculated.
5. **Obstacle Avoidance** → Adjusts path dynamically.
6. **Control & Execution** → Robot follows the planned route.

Create map -> localize yourself on the map -> select goal point -> use some algorithms to create routh -> -> `<loop>` GO -> check if everything goes fine -> correct yourself `</loop>`

+++++

1. Introduction to Navigation

Navigation in robotics involves moving from **point A to point B** while avoiding obstacles. In **ROS 2 and Gazebo (GZ) Harmonic**, this is typically handled using the **Navigation2 (Nav2) stack** combined with **sensor inputs, localization, mapping, and path planning**.

Key Components of Navigation:

1. **Perception** – Understanding the environment using cameras, LiDAR, sonar, etc.
2. **Localization** – Estimating the robot's position on a known or unknown map.
3. **Mapping** – Creating or using a map of the environment.
4. **Path Planning** – Finding an optimal and obstacle-free route.
5. **Control & Execution** – Converting plans into real movements via motor commands.

2. Simulation in Gazebo (GZ) Harmonic

Gazebo is a powerful **physics-based simulator** that allows robots to be tested in realistic environments before deploying them in the real world.

How Gazebo Works with ROS 2

- **Gazebo Sim (Harmonic) provides:**

1. Physics simulation for robot motion.
2. Plugins for sensors (cameras, LiDAR, sonar).
3. Integration with ROS 2 via [gazebo_ros_pkgs](#).

- **ROS 2 & Gazebo Workflow:**

1. **Define the Robot Model** (URDF/Xacro & SDF files).

2. **Add Sensors & Controllers** using Gazebo plugins.
 3. **Simulate & Test Navigation** with ROS 2 packages like Nav2 and SLAM.
-

3. Perception: Sensor Data Processing

For navigation, the robot requires **sensor data** to understand its surroundings.

Common Sensors Used in Navigation:

- **Cameras (RGB, Depth, Stereo)** – Used for **visual navigation and obstacle detection**.
- **LiDAR** – Generates **3D point clouds** for precise obstacle mapping.
- **Sonar & Ultrasonic Sensors** – Useful for **short-range obstacle detection**.
- **IMU (Inertial Measurement Unit)** – Provides **orientation and acceleration** data.

Processing Sensor Data in ROS 2:

- **Cameras:** `image_pipeline`, `cv_bridge` for OpenCV processing.
 - **LiDAR:** `lidar_ros_driver`, `pointcloud_to_laserscan`.
 - **Sensor Fusion:** `robot_localization` package combines **IMU, odometry, and GPS**.
-

4. Localization: Estimating Position

Localization helps the robot determine **where it is** on a map.

Types of Localization:

1. **Odometry-based Localization**
 - Uses **wheel encoders, IMU, and kinematics**.
 - Accumulates **error (drift) over time**.
2. **Monte Carlo Localization (MCL)**
 - Uses **Adaptive Monte Carlo Localization (AMCL)**.
 - Works with **pre-built maps** (requires an initial position estimate).
3. **SLAM (Simultaneous Localization and Mapping)**
 - Builds a map while **simultaneously localizing** the robot.
 - Used in **unknown environments**.

Localization Methods in ROS 2:

- **AMCL** (`nav2_amcl`) → Probabilistic particle filter for localization.
 - **SLAM Toolbox** (`slam_toolbox`) → Graph-based SLAM for real-time mapping.
 - **Visual SLAM (ORB-SLAM, RTAB-Map)** → Uses **camera-based** feature matching.
-

5. Mapping: Understanding the Environment

Mapping is essential for navigation in unknown environments.

Types of Maps Used in ROS 2:

- **2D Occupancy Grids** (Used for AMCL & Nav2)
- **3D Point Clouds** (For terrain and obstacle modeling)

Popular Mapping Techniques:

1. **GMapping** – Basic **2D SLAM** using LiDAR.
 2. **SLAM Toolbox** – **ROS 2-compatible 2D SLAM**.
 3. **RTAB-Map** – Supports **stereo and depth cameras** for 3D mapping.
-

6. Path Planning: Finding the Best Route

Once the robot knows where it is, it needs to plan a path to the goal.

Two Types of Path Planning:

- **Global Path Planning** – Finds an optimal path across a map.
- **Local Path Planning** – Adjusts the path dynamically to avoid obstacles.

Path Planning Algorithms in ROS 2:

- **A*** → Finds the **shortest path** in known maps.
- **Dijkstra** → Guarantees **optimal paths** but slower than A*.
- **RRT (Rapidly-exploring Random Trees)** → Works well in **dynamic or unknown environments**.

Local Planners in Nav2:

- **DWA (Dynamic Window Approach)** – Predicts **safe velocity-based paths**.
 - **TEB (Timed Elastic Band)** – Optimizes **paths with obstacle avoidance**.
-

7. Obstacle Avoidance: Real-Time Adjustments

The robot must detect and avoid obstacles **dynamically**.

Methods for Obstacle Avoidance:

1. **Reactive Methods**
 - Uses **sensor data (sonar, LiDAR)** to stop or adjust the path.
 - Example: Simple **threshold-based obstacle avoidance**.
2. **Vision-Based Obstacle Detection**
 - Uses **YOLO, OpenCV, or depth segmentation** to identify obstacles.
 - Can classify objects and plan around them.

3. Hybrid Approaches

- Combines **sonar (for close-range obstacles)** and **vision (for distant obstacles)**.
 - Uses **sensor fusion** for better decision-making.
-

8. Control & Execution: Moving the Robot

Once a path is planned, the robot needs to **follow it precisely**.

Locomotion Control for a Hexapod:

- **Inverse Kinematics (IK)** – Computes joint angles for each leg movement.
- **Central Pattern Generators (CPG)** – Bio-inspired walking behavior.
- **Model Predictive Control (MPC)** – Optimizes movement for stability.

Trajectory Following Algorithms:

- **PID (Proportional-Integral-Derivative)** – Basic but effective control loop.
 - **MPC (Model Predictive Control)** – Uses a predictive model for smooth motion.
 - **Reinforcement Learning (RL)** – AI-based adaptation to terrain.
-

9. Advanced Topics in Navigation

Multi-Robot Navigation

- Uses **ROS 2 multi-node communication** for **fleet coordination**.
- Requires **distributed path planning** (e.g., Swarm Robotics).

Dynamic Environments

- *RRT-Smart & AI-based planners** adapt to **changing obstacles**.
- **Deep Learning models** predict **pedestrian movement** for safer navigation.

Rough Terrain Navigation (Hexapods, Legged Robots)

- **Terrain Classification** using **vision & depth sensors**.
 - **Gait Adaptation** based on **real-time feedback**.
 - **Bio-inspired approaches** for stable walking.
-

10. Conclusion: How Everything Fits Together

7. **Perception** → Sensors provide data.
8. **Localization** → The robot determines its position.
9. **Mapping** → The environment is understood.
10. **Path Planning** → A safe route is calculated.
11. **Obstacle Avoidance** → Adjusts path dynamically.
12. **Control & Execution** → Robot follows the planned route.

Advanced Topics in Navigation with ROS 2 & Gazebo

Now that we've covered the fundamentals, let's dive into **advanced topics** in navigation for more complex and dynamic environments. These include **multi-robot coordination**, **dynamic path planning**, **reinforcement learning-based navigation**, **terrain adaptation**, and **bio-inspired locomotion for legged robots** like hexapods.

1. Multi-Robot Navigation & Swarm Coordination

When multiple robots operate in the same space, they need to **coordinate movements** to avoid collisions and optimize efficiency.

Challenges in Multi-Robot Navigation

- **Collision Avoidance** → Robots must plan paths dynamically to prevent crashes.
- **Communication Overhead** → Robots must share their positions and intentions.
- **Task Allocation** → Assigning robots to different areas or goals optimally.

Approaches in ROS 2 for Multi-Robot Navigation

1. **Multi-Robot Nav2 Instances**
 - Each robot runs its own **Nav2 instance** with unique namespaces.
 - Example: `robot_1/nav2_controller`, `robot_2/nav2_controller`.
2. **Centralized Planning with Traffic Control**
 - A global planner assigns paths to each robot.
 - Uses **Conflict-Based Search (CBS)** or **Priority Graphs** to manage robot motion.
3. **Decentralized Swarm Behavior (Bio-Inspired Methods)**
 - **Boid Algorithms**: Flocks of robots move like birds/swarm intelligence.
 - **Ant Colony Optimization (ACO)**: Robots "pheromone" mark areas they have explored.

Key ROS 2 Packages for Multi-Robot Systems

- **multi_robot_navigation** → Implements centralized & decentralized coordination.
 - **ros2_multirobot_map_merge** → Merges individual robot maps into a shared global map.
 - **Swarm ROS 2** → Implements bio-inspired swarm algorithms.
-

2. Dynamic Path Planning & Adaptive Navigation

Traditional planners like A* assume a **static environment**, but real-world conditions **change continuously**.

Advanced Path Planning Techniques

1. *RRT & Informed RRT**

- Improves on RRT by optimizing paths.
 - Good for **dynamic and unknown environments**.
2. *Hybrid A (HA)***
- A* combined with **kinematic constraints**.
 - Used in **car-like or legged robots**.
3. **Graph Neural Networks (GNN) for Path Planning**
- Uses AI to optimize paths dynamically.
 - Can learn **traffic conditions** in real-time.
4. **Trajectory Optimization with MPC**
- Model Predictive Control (MPC) optimizes motion dynamically.
 - Uses physics models to compute **minimum-energy movement**.

ROS 2 Implementations

- **teb_local_planner** → Optimizes **non-holonomic motion constraints**.
 - **mpc_local_planner** → Uses **MPC for trajectory optimization**.
 - **rl_nav** → Uses **reinforcement learning** for navigation.
-

3. Reinforcement Learning (RL) in Navigation

Traditional path planners rely on **explicit algorithms**, but RL allows robots to **learn and adapt** to environments over time.

RL vs. Traditional Navigation

Method	Pros	Cons
A* / Dijkstra	Fast, deterministic	Fails in unknown terrain
RRT*	Handles dynamic spaces	Slow in real-time
RL (PPO, DDPG)	Learns terrain, adaptive	Needs training, high computation

Deep Reinforcement Learning for Navigation

- **Proximal Policy Optimization (PPO)** → Trains robots to navigate by rewards.
- **Deep Deterministic Policy Gradient (DDPG)** → Works in **continuous action spaces** (good for hexapods).
- **SAC (Soft Actor-Critic)** → Balances **exploration and exploitation** for optimal motion.

ROS 2 Implementations

- **nav2_behavior_trees with RL** → Uses RL-trained models inside the Nav2 framework.
 - **ros2_reinforcement_learning** → Implements **PPO/DDPG for robotics**.
 - **OpenAI Gym + Gazebo** → Custom RL training environments.
-

4. Terrain Adaptation & Rough Terrain Navigation

Hexapods must **adjust their gait dynamically** based on terrain conditions.

Terrain Classification & Sensing

1. Vision-Based Terrain Classification

- Uses **CNNs (Convolutional Neural Networks)** to detect obstacles, stairs, slopes.
- Example: **YOLOv8 for terrain recognition**.

2. Depth-Based Terrain Mapping

- Uses **LiDAR or stereo cameras** to generate a **heightmap**.
- Allows real-time slope detection.

3. Force Feedback from Legs

- Sensors on legs measure **ground contact force**.
- Adjusts walking style based on detected surface hardness.

Adaptive Locomotion Strategies

- **CPG (Central Pattern Generators)** → Simulates biological **gait adaptation**.
- **Inverse Reinforcement Learning (IRL)** → Learns optimal foot placements based on experience.
- **Online Gait Optimization (Bayesian Optimization)** → Continuously adjusts step size, height.

ROS 2 Packages for Terrain Adaptation

- **ros2_terrain_classification** → Uses CNN-based **real-time terrain recognition**.
 - **hexapod_gait_controller** → Implements **CPG-based walking for hexapods**.
 - **ros2_dynamic_footstep_planner** → Adjusts **foot placements dynamically**.
-

5. Bio-Inspired Locomotion for Hexapods

Hexapods **mimic insect movement**, but advanced methods improve their efficiency.

Bio-Inspired Control Methods

1. Spiking Neural Networks (SNNs)

- Mimics **biological neurons** for movement.
- Can self-adjust for **energy-efficient walking**.

2. Neuroevolution (Evolutionary Robotics)

- Uses **genetic algorithms (GA)** to evolve better walking styles.

3. Dynamic Stability Models

- Uses **Zero Moment Point (ZMP)** or **Capture Point (CP)** for stability in **rough terrain**.

ROS 2 & Gazebo Implementations

- **ros2_neuro_gait** → Uses **SNNs for adaptive locomotion**.

- **hexapod_simulation_rl** → Uses **GA** for walking pattern evolution.
 - **ZMP-based Hexapod Stability** → Implements **real-time balance correction**.
-

6. Human-Robot Interaction & High-Level Decision Making

Hexapods often **work alongside humans** in exploration, rescue, or industry.

Human-in-the-Loop Navigation

- **Gesture Control** → Uses **vision-based hand-tracking**.
- **Brain-Computer Interface (BCI)** → EEG-based control.
- **Speech Navigation** → NLP-based command execution.

Hierarchical Decision Making

1. **High-Level Task Planner** → Uses **PDDL (Planning Domain Definition Language)** for complex tasks.
2. **Mid-Level Navigation** → Uses **Nav2 with custom behavioral constraints**.
3. **Low-Level Locomotion** → Executes movement using **inverse kinematics & RL-based controllers**.

ROS 2 Implementations

- **ros2_voice_nav** → Uses NLP for speech-controlled navigation.
- **ros2_human_tracking** → Detects and follows a human in real time.
- **pddl_ros2_planner** → Implements **task-level AI decision making**.

Summary of Algorithms for Trajectory Planning

Trajectory planning (trace planning) is crucial for **smooth and efficient robot movement**. Below is a categorized summary of **deterministic, sampling-based, optimization-based, and AI-driven** algorithms used in ROS 2 for trajectory planning.

1. Deterministic Path Planning Algorithms

These algorithms generate a fixed path based on a known environment.

A (A-Star)*

- **How it works:** Expands nodes using a **cost function** combining distance and heuristics.
- **Pros:** Guarantees shortest path, works in grid-based maps.
- **Cons:** Not ideal for real-time dynamic environments.
- **Used in ROS 2:** `nav2_planner`, `global_planner`.

Dijkstra's Algorithm

- **How it works:** Similar to A* but searches all possible paths without heuristics.
- **Pros:** Finds the absolute shortest path.
- **Cons:** Computationally expensive.

- Used in ROS 2: `dijkstra_global_planner`.

Hybrid A*

- **How it works:** A* adapted for non-holonomic robots, considers vehicle kinematics.
 - **Pros:** Better for wheeled robots and legged robots requiring smooth turns.
 - **Cons:** More complex computation.
 - Used in ROS 2: `hybrid_astar_planner`.
-

2. Sampling-Based Motion Planning Algorithms

Useful for **complex, high-dimensional spaces**, these algorithms **randomly sample paths** and refine them.

RRT (Rapidly-exploring Random Tree)

- **How it works:** Randomly samples points and expands a tree from the start position.
- **Pros:** Fast, works well in high-dimensional space.
- **Cons:** Paths need smoothing for real-world use.
- Used in ROS 2: `ompl_planner` (Open Motion Planning Library).

RRT*

- **How it works:** An improved RRT that **optimizes paths** as the tree expands.
- **Pros:** Finds shorter paths than RRT, reduces unnecessary detours.
- **Cons:** Slower than standard RRT.
- Used in ROS 2: `ompl_planner`, `moveit_planners_ompl`.

PRM (Probabilistic Roadmap Method)

- **How it works:** Constructs a **graph of random points** in free space and connects them.
 - **Pros:** Efficient for **multi-query planning** in static environments.
 - **Cons:** Less effective in dynamic environments.
 - Used in ROS 2: `ompl_planner`.
-

3. Optimization-Based Trajectory Planning

Optimization-based planners refine the trajectory to **minimize energy, time, or jerk (sudden changes in motion)**.

TEB (Timed Elastic Band)

- **How it works:** Optimizes trajectories by minimizing time while considering obstacles.
- **Pros:** Dynamic obstacle avoidance, non-holonomic constraints.
- **Cons:** Requires good initial path estimation.
- Used in ROS 2: `teb_local_planner`.

CHOMP (Covariant Hamiltonian Optimization for Motion Planning)

- **How it works:** Uses gradient-based optimization to smooth trajectories.
- **Pros:** Works well in **high-dimensional motion planning** (e.g., robotic arms, hexapods).

- **Cons:** Computationally expensive.
- **Used in ROS 2:** `chomp_motion_planner` in MoveIt!.

STOMP (Stochastic Trajectory Optimization for Motion Planning)

- **How it works:** Uses **stochastic gradient descent** to improve paths.
- **Pros:** No need for explicit gradient computation.
- **Cons:** May get stuck in local minima.
- **Used in ROS 2:** `stomp_motion_planner` in MoveIt!.

Model Predictive Control (MPC)

- **How it works:** Solves an **optimization problem at each step**, predicting future motion.
 - **Pros:** Excellent for real-time, smooth trajectory execution.
 - **Cons:** Requires high computation power.
 - **Used in ROS 2:** `mpc_local_planner`.
-

4. AI-Based & Learning-Based Trajectory Planning

Machine learning and reinforcement learning (RL) approaches allow robots to **adapt to environments** without explicit path programming.

Deep Reinforcement Learning (RL) for Navigation

- **How it works:** The robot learns a policy by interacting with the environment.
- **Pros:** Adapts to unknown environments, works in real-time.
- **Cons:** Requires extensive training in a simulator.
- **Used in ROS 2:** `ros2_reinforcement_learning`, OpenAI Gym + Gazebo.

Imitation Learning (IL) for Trajectory Generation

- **How it works:** Trains robots using **demonstrated human/robot movements**.
- **Pros:** Learns from expert demonstrations, intuitive.
- **Cons:** Limited generalization to unseen environments.
- **Used in ROS 2:** `ros2_imitation_learning`.

Graph Neural Networks (GNN) for Path Planning

- **How it works:** Uses **graph representations of the environment** for motion planning.
 - **Pros:** Can predict **dynamic obstacles** and optimize routes.
 - **Cons:** Requires pre-training on large datasets.
 - **Used in ROS 2:** Custom implementations in PyTorch/TensorFlow with ROS 2.
-

5. Multi-Agent and Swarm-Based Planning

For multiple robots or **swarm robotics**, traditional planners don't scale well.

Conflict-Based Search (CBS)

- **How it works:** Plans paths for each robot and resolves conflicts dynamically.
- **Pros:** Scales well for **multi-robot path planning**.

- **Cons:** Requires significant computation for large swarms.
- **Used in ROS 2:** `multi_agent_planner`.

Ant Colony Optimization (ACO)

- **How it works:** Robots use a pheromone-based approach to mark optimal paths.
- **Pros:** Works well for **exploration and coverage tasks**.
- **Cons:** Convergence time can be long.
- **Used in ROS 2:** `swarm_ros2`.

Bio-Inspired Neural Path Planning

- **How it works:** Uses **spiking neural networks (SNNs)** to generate adaptive paths.
- **Pros:** Mimics **biological movement strategies**, highly adaptable.
- **Cons:** Computationally expensive.
- **Used in ROS 2:** `ros2_bio_nav`.

Choosing the Right Algorithm for Your Use Case

Use Case	Recommended Algorithm
Static Environment, Shortest Path	A*, Dijkstra, Hybrid A*
Dynamic Environments	RRT*, TEB, MPC
High-Dimensional Motion Planning	CHOMP, STOMP
Multi-Robot Coordination	CBS, PRM, ACO
Reinforcement Learning-Based	PPO, DDPG, SAC
Adaptive & Bio-Inspired Movement	Spiking Neural Networks, CPG

List of ROS & Gazebo Functions and Packages Mentioned

Core ROS 2 Navigation Functions & Packages

- **Nav2 (Navigation2 Stack)**
- `nav2_planner`
- `nav2_amcl`
- `nav2_behavior_trees`
- `nav2_controller`
- `global_planner`
- `dijkstra_global_planner`
- `hybrid_astar_planner`
- `teb_local_planner`
- `mpc_local_planner`
- `stomp_motion_planner`

- `chomp_motion_planner`
- `slam_toolbox`
- `robot_localization`
- `multi_robot_navigation`
- `ros2_multirobot_map_merge`

Gazebo Integration & Plugins

- **Gazebo Sim (Harmonic)**
- `gazebo_ros_pkgs`
- **URDF/Xacro** (for defining robot models)
- **SDF files** (for simulation models)

Perception & Sensor Processing

- `image_pipeline`
- `cv_bridge`
- `lidar_ros_driver`
- `pointcloud_to_laserscan`

Path Planning Algorithms & Implementations

- `ompl_planner`
- `moveit_planners_ompl`
- `rl_nav`
- `swarm_ros2`
- `multi_agent_planner`
- `ros2_bio_nav`

Reinforcement Learning & AI-Based Navigation

- `ros2_reinforcement_learning`
- `ros2_imitation_learning`
- `OpenAI Gym + Gazebo`

Terrain Adaptation & Bio-Inspired Locomotion

- `ros2_terrain_classification`
- `hexapod_gait_controller`
- `ros2_dynamic_footstep_planner`
- `ros2_neuro_gait`
- `hexapod_simulation_rl`
- `ZMP-based Hexapod Stability`

Human-Robot Interaction & Decision Making

- `ros2_voice_nav`
- `ros2_human_tracking`
- `pddl_ros2_planner`